



THE UNIVERSITY OF
AUCKLAND
Te Whare Wānanga o Tāmaki Makaurau
NEW ZEALAND

Welcome to COMPSCI 703 Generalising Artificial Intelligence

Dr Hong Jia
University of Auckland

Compound AI Systems

Impressive AI demos

when was University of Auckland founded?



The University of Auckland was founded in **1883**. [University of ...](#) +1

More precisely, it was formally opened on **23 May 1883** as *Auckland University College*, originally part of the University of New Zealand. [en.wikipedia.org](#)

     ...    Sources

“Parallelize this loop nest with OpenMP using 16 threads, and explain whether the chosen schedule is appropriate.”

C



```
for (int t = 1; t < T; t++) {  
    for (int i = 1; i < N - 1; i++) {  
        next[i] = 0.25 * cur[i - 1]  
                + 0.5 * cur[i]  
                + 0.25 * cur[i + 1];  
    }  
  
    for (int i = 1; i < N - 1; i++) {  
        cur[i] = next[i];  
    }  
}
```

Reliable AI Systems

Question: In Fibonacci sequence, it follows the rule that each number is equal to the sum of the preceding two numbers. Assuming the first two numbers are 0 and 1, what is the 50th number in Fibonacci sequence?

The first number is 0, the second number is 1, therefore, the third number is $0+1=1$. The fourth number is $1+1=2$. The fifth number is $1+2=3$. The sixth number is $2+3=5$. The seventh number is $3+5=8$. The eighth number is $5+8=13$.
..... (Skip 1000 tokens)
The 50th number is 32,432,268,459.

CoT

32,432,268,459



```
length_of_fibonacci_sequence = 50
fibonacci_sequence = np.zeros(length_of_)
fibonacci_sequence[0] = 0
fibonacci_sequence[1] = 1
For i in range(3, length_of_fibonacci_sequence):
    fibonacci_sequence[i] = fibonacci_sequence[i-1] +
    fibonacci_sequence[i-2]
ans = fibonacci_sequence[-1]
```

PoT



12,586,269,025



ceo of air nz

As of **June 2024**, the CEO of Air New Zealand is **Greg Foran**. He has held the position since February 2020. Please note that executive



ceo of air nz

AI Mode All Images News Videos Forums Shopping More ▾ Tools ▾

◆ AI Overview

Nikhil Ravishankar became the Chief Executive Officer of Air New Zealand in October 2025, succeeding Greg Foran. Previously the airline's Chief Digital Officer since 2021, Ravishankar has focused on digital transformation, technology advances, and navigating industry disruptions. [✦ Air New Zealand +1](#)

Turning monolithic LMs into reliable AI systems remains challenging.

Reliable AI Systems



[Home](#) [News](#) [Sport](#) [Business](#) [Technology](#) [Health](#) [Culture](#) [Arts](#) [Travel](#) [Earth](#) [Audio](#) [Video](#) [Live](#)

Airline held liable for its chatbot giving passenger bad advice - what this means for travellers

Air Canada had to pay Moffatt \$812.02 (£642.64) in damages and tribunal fees. "It should be obvious to Air Canada that it is responsible for all the information on its website,"

Two US lawyers fined for submitting fake court citations from ChatGPT

Law firm also penalised after chatbot invented six legal cases that were then used in an aviation injury claim

Reliable AI Systems



Every AI system will make mistakes.

But the **monolithic** nature of LMs makes them hard to **control, debug, and improve**.

Compound AI Systems



To tackle this, AI researchers increasingly build Compound AI Systems.

Compound AI Systems:

modular programs that use LMs as specialized components

Compound AI Systems

i.e. modular programs that use LMs as specialized components



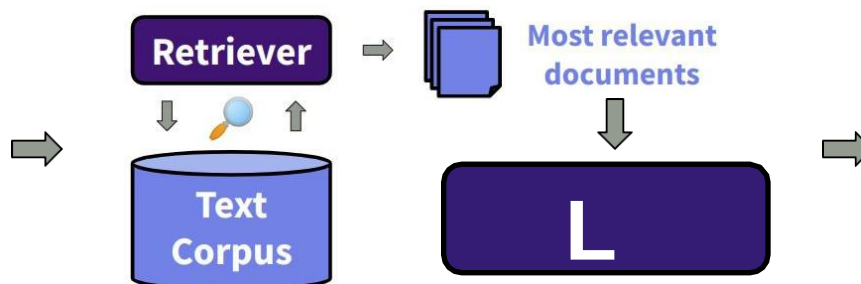
Literature: DrQA (Chen et al., 2017), ORQA (Lee et al., 2019), RAG (Lewis et al., 2020), ColBERT-QA (Khattab et al., 2020)

Compound AI Systems

i.e. modular programs that use LMs as specialized components

Example: Retrieval-Augmented Generation

What
compounds
protect the
digestive
system?



The stomach is
protected by
gastric acid and
proteases.



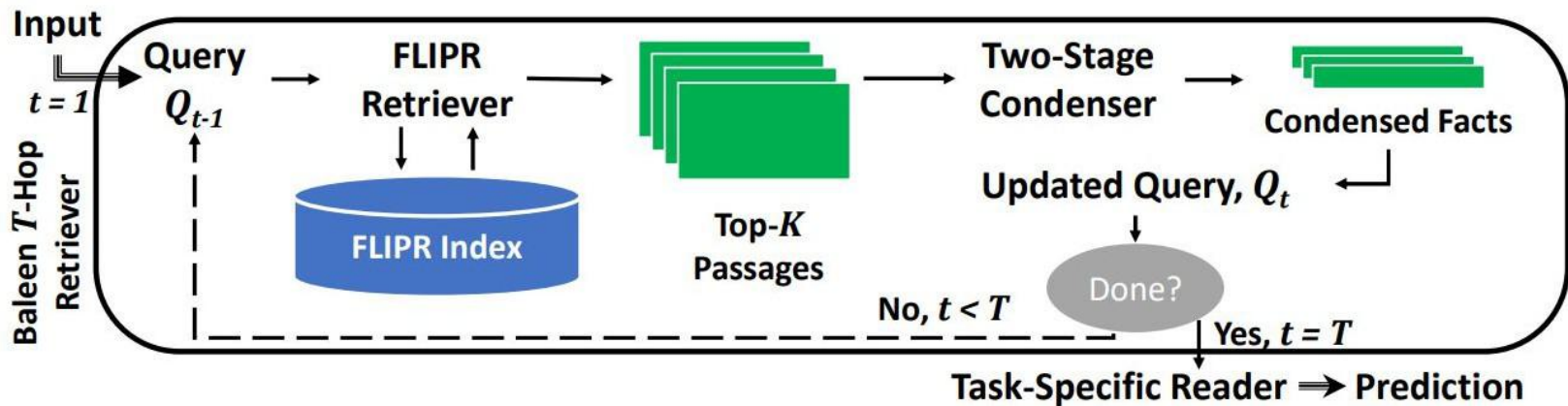
Transparency: can debug traces & offer user-facing attribution

Efficiency: can use smaller LMs, offloading knowledge & control flow

Compound AI Systems

i.e. modular programs that use LMs as specialized components

Example: Multi-Hop Retrieval-Augmented Generation

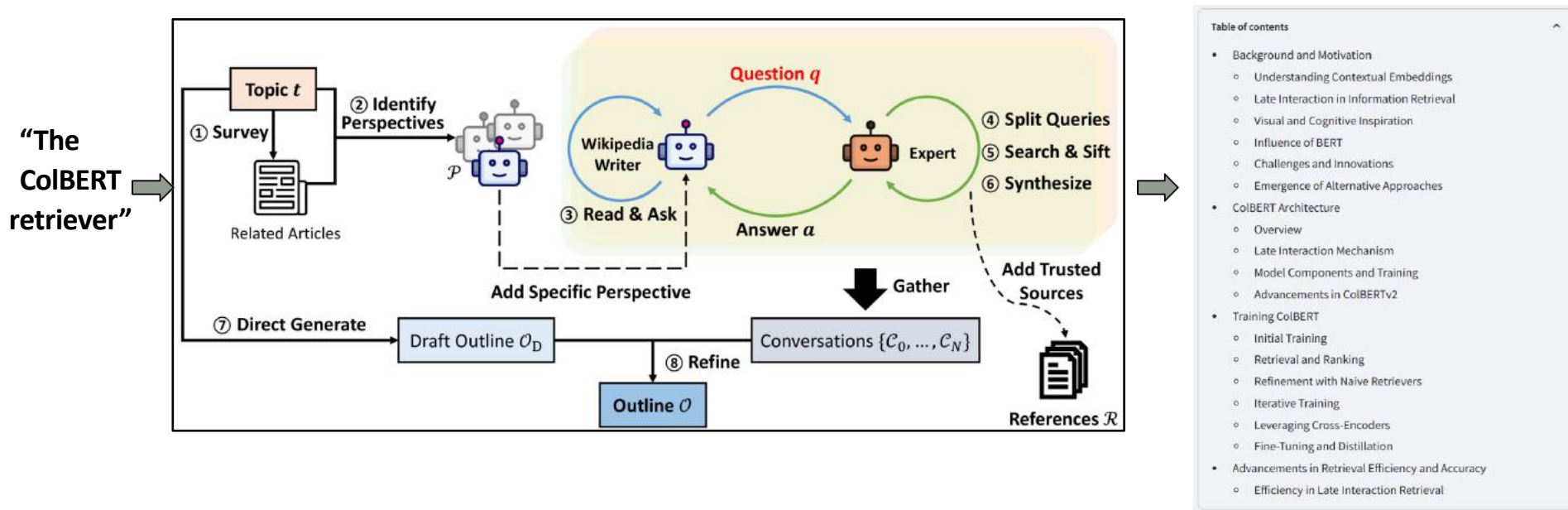


⚡ Control: can iteratively improve the system & ground it via tools

Literature: GoldEn (Qi et al., 2019), DecompRC (Min et al., 2019), MDR (Xiong et al., 2020), Baleen (Khattab et al., 2021)

Compound AI Systems

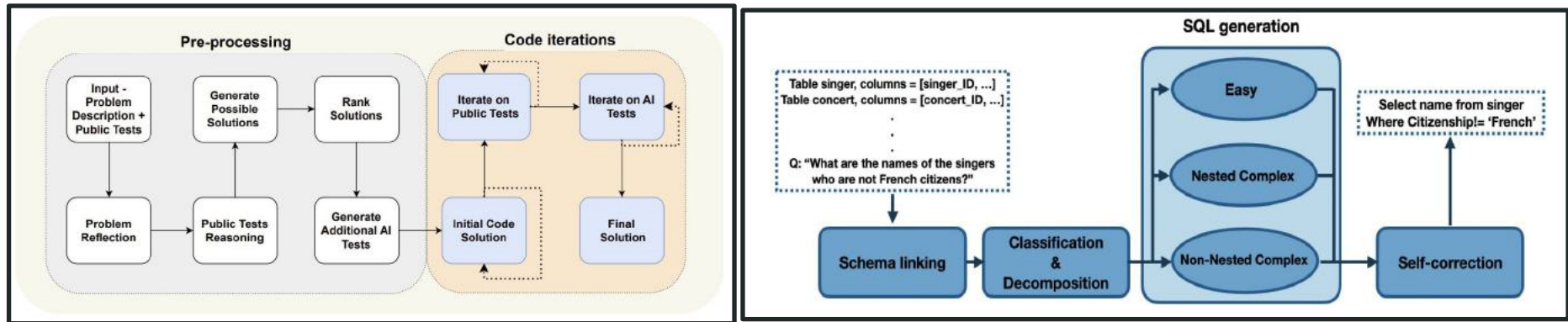
Example: Compositional Report Generation,
i.e. brainstorming an outline, collecting references, etc.



⚡ Quality: more reliable composition of better-scoped LM capabilities

Compound AI Systems

i.e. modular programs that use LMs as specialized components



+Task-agnostic prompting strategies, e.g. Best-of-N, Chain Of Thought, Program of Thought, ReAct, Reflexion, Archon, ...

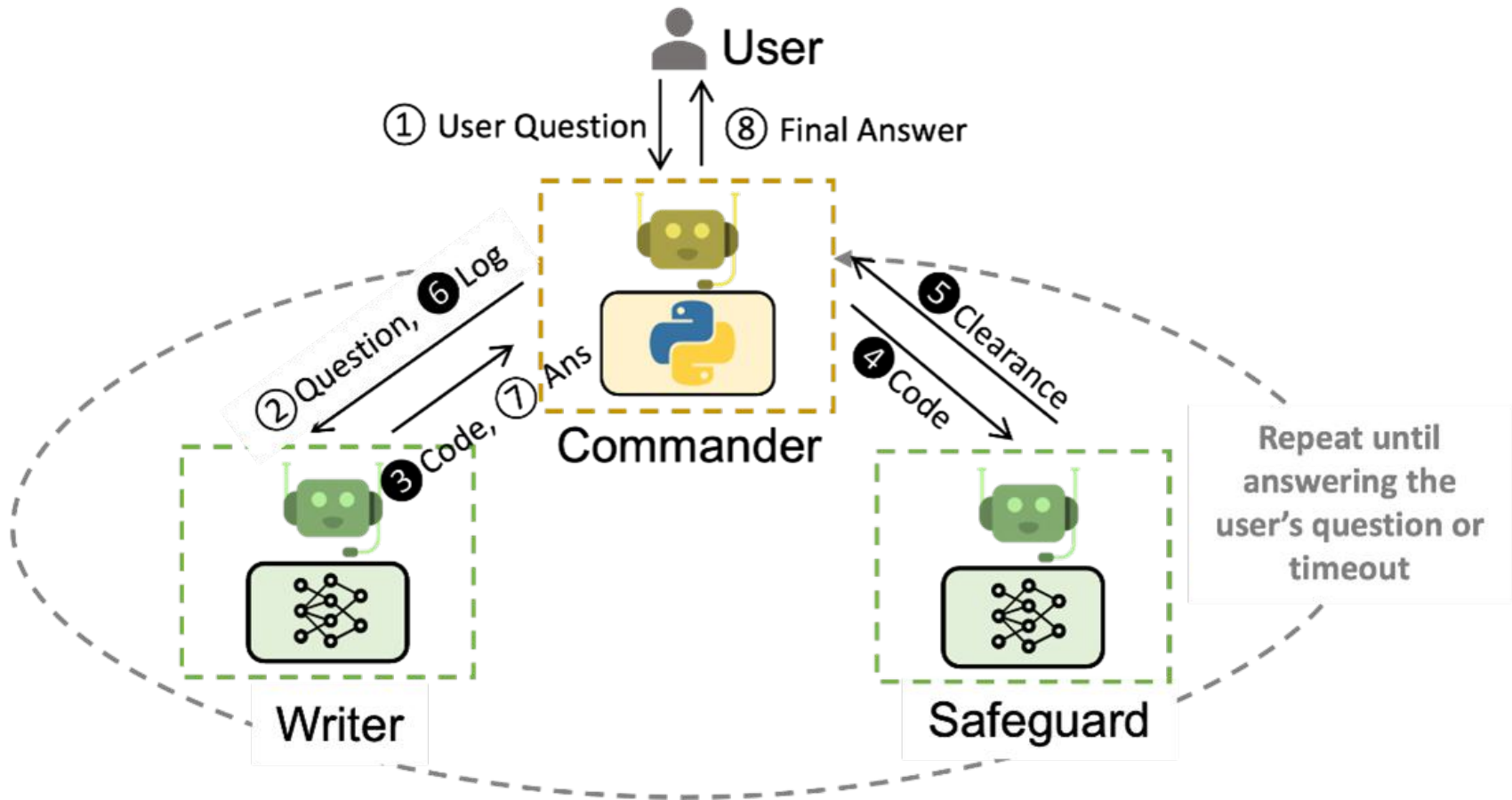
⚡ Inference-time Scaling: systematically searching for better outputs

Why Compound AI Systems?



1. Quality: more reliable composition of better-scoped LM capabilities
2. Control: can iteratively improve the system & ground it via tools
3. Transparency: can debug trajectories & offer user-facing attribution
4. Efficiency: can use smaller LMs, offloading knowledge & control flow
5. Inference-time Scaling: can systematically search for better outputs

Recap



Prompting...

```
1 [web] I will think step by step and answer your question.
2
3 Question: is growing seedless cucumber good for a gardener with entomophobia
4 Explanation: Entomophobia is a fear of insects. Plants need insects to pollinate them. Seedless fruits such as seedless
5 cucumbers do not require pollination, so seedless fruits do not require insects. This makes good for people with
6 entomophobia.
7 Answer: Yes
8
9 Question: Who was british pm and viceroy during quit india movement?
10 Explanation: The Quit India Movement was launched in 8th August 1942. The british PM at that time was Winston Churchill.
11 The british viceroy during the movement is Victor Hope, usually referred to as Lord Linlithgow.
12 Answer: Winston Churchill and Lord Linlithgow
13
14 Question: Which year does game over man come out on netflix?
15 Explanation: Game Over, Man! is an action-comedy movie released in March 23, 2018, on Netflix. It's director is Kyle
16 Newacheck. The movie stars Anders Holm, Adam DeVine, and Blake Anderson.
17 Answer: March 23, 2018
18
19 Question: would it be very difficult for Nuno Gomes to dive to the Red Sea's deepest point?
20 Explanation: Nuno Gomes' deepest dive in the Red Sea to date is 317 metres. The Red Sea has a maximum depth of over 3,000
21 metres. So it would be difficult for Nuno Gomes to the deepest point of the Red sea.
22 Answer: Yes
23
24 Question: Are chinchillas cold-blooded?
25 Explanation: Chinchillas are rodents, which are mammals. All mammals are warm-blooded.
26 Answer: No
27
```

J.5. Object Counting

```
# Q: I have a chair, two potatoes, a cauliflower, a lettuce head, two tables, a
  cabbage, two onions, and three fridges. How many vegetables do I have?

# note: I'm not counting the chair, tables, or fridges
vegetables_to_count = {
    'potato': 2,
    'cauliflower': 1,
    'lettuce head': 1,
    'cabbage': 1,
    'onion': 2
}

print(sum(vegetables_to_count.values()))

# Q: I have a drum, a flute, a clarinet, a violin, four accordions, a piano, a
  trombone, and a trumpet. How many musical instruments do I have?

musical_instruments_to_count = {
    'drum': 1,
    'flute': 1,
    'clarinet': 1,
    'violin': 1,
    'accordion': 4
}
```

Code

Blame

1 lines (1 loc) · 60.9 KB

```
1 {"react_put_0": "You are in the middle of a room. Looking quickly
```

Unfortunately, LMs are **highly sensitive** to how they're instructed to solve tasks, so under the hood we often...

Prompting...

The usual way of writing prompts is:

- you manually write a long set of instructions
- you try few-shot examples
- you keep revising the wording
- you tune it based on experience and intuition

....

The problems

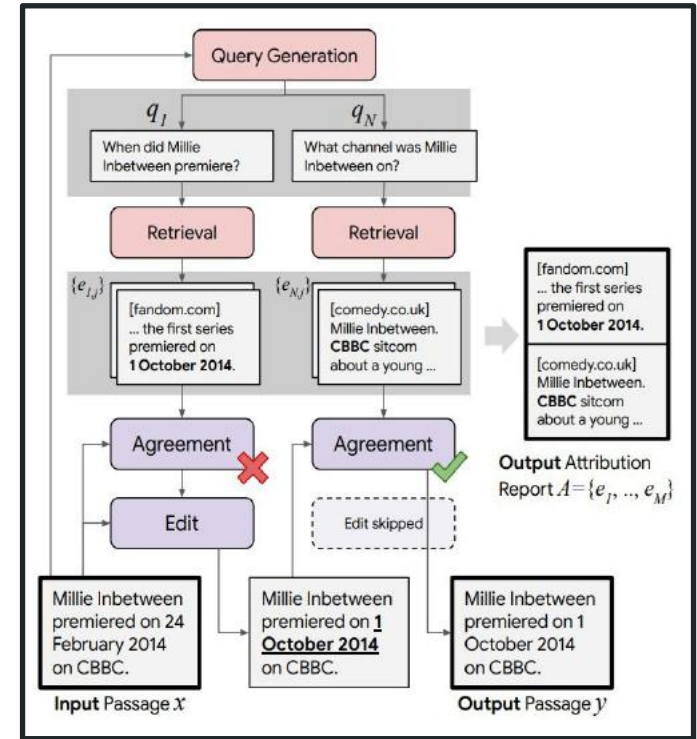
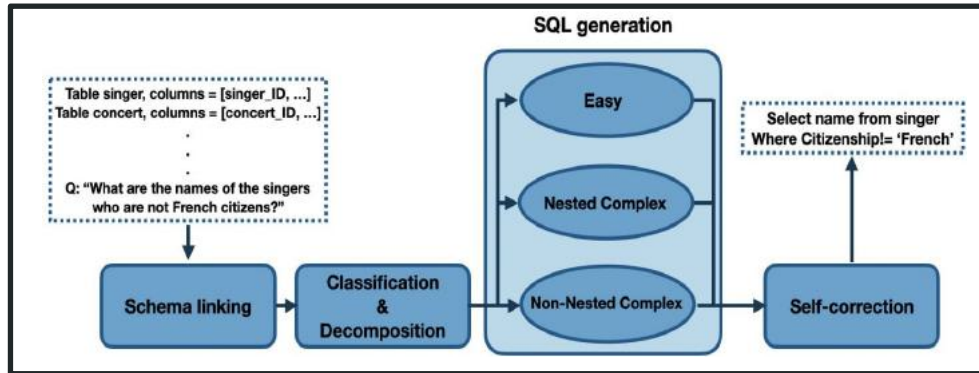
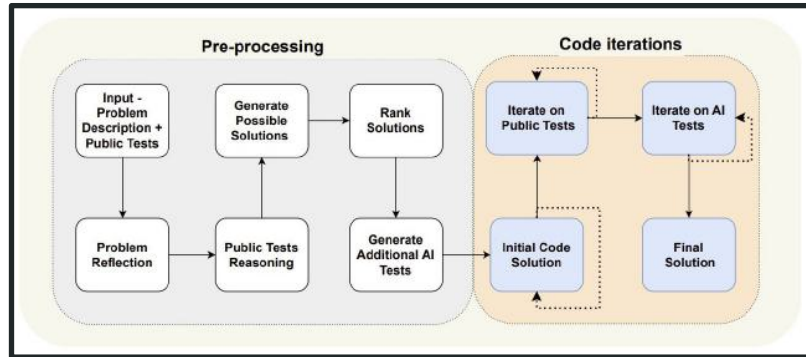
Each “prompt” couples five very different roles:

- The core input → output behavior, a **Signature**.
- The computation specializing an inference-time strategy to the signature, a **Predictor**.
- The computation formatting the signature’s inputs and parsing its typed outputs, an **Adapter**.
- The computations defining objectives and constraints on behavior, **Metrics and Assertions**.
- The strings that instruct (or weights that adapt) the LM for desired behavior, an **Optimizer**.
- *Existing Compound AI Systems are modular in principle, but are too “stringly-typed”: they couple the fundamental system architecture with incidental choices not portable to new LMs, objectives, or pipelines.*

Compound AI Systems

- We know how to build controllable systems & improve them modularly.
- That is called... **programming**.
- What if we could abstract Compound AI Systems as programs with fuzzy natural-language-typed modules that learn their behavior?

DSPy Framework



LM program $\Phi : \mathcal{X} \rightarrow \mathcal{Y}$, with $\mathcal{X}$ and $\mathcal{Y}$ in natural language.

In the course of its execution, Φ makes calls to modules $\langle M_1, \dots, M_{|M|} \rangle$.

Each module $M_i : \mathcal{X}_i \rightarrow \mathcal{Y}_i$ is a *declarative* LM invocation, defined via inherently fuzzy natural-language descriptions of: (1) a sub-task $\mathcal{D}_i^T$ (optional), (2) input domain type(s) $\mathcal{D}_i^X$, and (3) output co-domain type(s) $\mathcal{D}_i^Y$.

DSPy Framework



```
fact_checking = dspy.ChainOfThought('claims -> verdicts: list[bool]')
fact_checking(claims=["Python was released in 1991.", "Python is a compiled language."])
```

Prediction(

reasoning='The first claim states that "Python was released in 1991," which is true. Python was indeed first released by Guido van Rossum in February 1991. The second claim states that "Python is a compiled language." This is false; Python is primarily an interpreted language, although it can be compiled to bytecode, it is not considered a compiled language in the traditional sense like C or Java.'

verdicts=[True, False]

)

- What if we want to optimize the LMs of each module?

DSPy Framework



DSPy doesn't ask you to hard-code a long prompt text. Instead, it breaks the task down into:

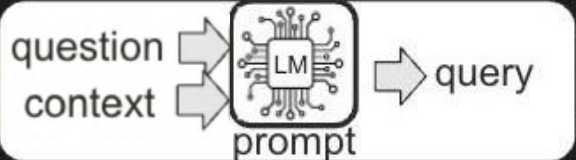
- what the inputs are
- what the intermediate steps are
- what the output format is
- which module to call at each step
- how to automatically optimize based on metrics

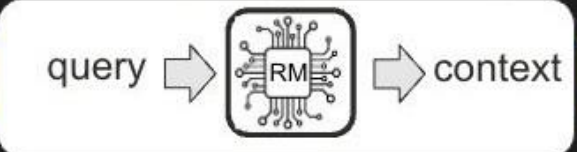
Example

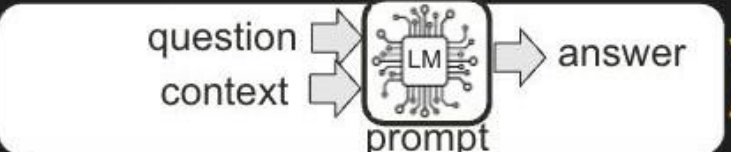
As an example, let's say we wanted to build this simple pipeline for multi-hop retrieval-augmented generation

```
def multihop_qa(question:str) -> str:
```

```
    for i in range(2):
```

```
        query = (  )
```

```
        context = (  )
```

```
    return (  )
```

Example

```
class MultiHop(dspy.Module):
    def __init__(self):
        self.generate_query = dspy.ChainOfThought("context, question -> query")
        self.generate_answer = dspy.ChainOfThought("context, question -> answer")
    def forward(self, question):
        context = []
        for hop in range(2):
            query = self.generate_query(context, question).query
            context += dspy.Retrieve(k=3)(query).passages
        answer = self.generate_answer(context, question)
        return answer
```

Example

```
class MultiHop(dspy.Module):  
    def __init__(self):  
        self.generate_query = dspy.ChainOfThought("context, question -> query")  
        self.generate_answer = dspy.ChainOfThought("context, question -> answer")  
    def forward(self, question):  
        context = []  
        for hop in range(2):  
            query = self.generate_query(context, question).query  
            context +=  
                dspy.Retrieve(k=3)(query).passages  
        answer = self.generate_answer(context, question)  
        return answer
```

Init method defines LM calls

Example

```
class MultiHop(dspy.Module):
    def __init__(self):
        self.generate_query = dspy.ChainOfThought("context, question -> query")
        self.generate_answer = dspy.ChainOfThought("context, question -> answer")
    def forward(self, question):
        context = []
        for hop in range(2):
            query = self.generate_query(context, question).query
            context += dspy.Retrieve(k=3)(query).passages
        answer = self.generate_answer(context, question)
        return answer
```

Forward method defines program logic

Example

```
class MultiHop(dspy.Module):
    def __init__(self):
        self.generate_query = dspy.ChainOfThought("context, question -> query")
        self.generate_answer = dspy.ChainOfThought("context, question -> answer")
    def forward(self, question):
        context = []
        for hop in range(2):
            query = self.generate_query(context, question).query
            context += dspy.Retrieve(k=3)(query).passages
        answer = self.generate_answer(context, question)
        return answer
```

Signature: what to do, not how to prompt!

Example

```
class MultiHop(dspy.Module):
    def __init__(self):
        self.generate_query = dspy.ChainOfThought("context, question -> query")
        self.generate_answer = dspy.ChainOfThought("context, question -> answer")
    def forward(self, question):
        context = []
        for hop in range(2):
            query = self.generate_query(context, question).query
            context += dspy.Retrieve(k=3)(query).passages
        answer = self.generate_answer(context, question)
        return answer
```

**Modules define the strategy
for expressing a signature**

Example

```
class MultiHop(dspy.Module):
    def __init__(self):
        self.generate_query = dspy.ChainOfThought("context, question -> query")
        self.generate_answer = dspy.ChainOfThought("context, question -> answer")
    def forward(self, question):
        context = []
        for hop in range(2):
            query = self.generate_query(context, question).query
            context += dspy.Retrieve(k=3)(query).passages
        answer = self.generate_answer(context, question)
        return answer
```

How can we translate these into high-quality prompts?

Example

First, modules are **translated into basic prompts** using Adapters and Predictors.

```
self.generate_query = dspy.ChainOfThought("context, question -> query")
```

```
dspy.Adapter(self.generate_query)
```



**Predefined Adapters are used to
translate modules into basic prompts**

Given the fields "context" and "question", respond with the field "query".

Follow the following format:

Context: <context>

Question: <question>

Reasoning: Let's think step by step to <..>

Query: <query>

Example

DSPy's **Optimizers** can then tune this prompt!

... jointly along with all other prompts in your program

Given the fields "context" and "question", respond with the field "query".

Follow the following format:

Context: <context> Question: <question>

Reasoning: Let's think step by step to <..>

Query: <query>

Program Score: 37%

optimizer = MIPROv2()

optimized_program = optimizer.compile(program)



Carefully read the provided `context` and `question`. Your task is to formulate a concise and relevant `query` that could be used to retrieve information from a search engine to answer the question most effectively. The `query` should encapsulate...

Follow the following format:

Context: <context> Question: <question>

Reasoning: Let's think step by step to <..>

Query: <query>

Here are some examples: <...>

Program Score: 55%

Example

Instead of tweaking a string prompt...

Solve a question answering task with interleaving Thought, Action, Observation steps. Thought can reason about the current situation, and Action can be three types:

- (1) Search[entity], which searches the exact entity on Wikipedia and returns the first paragraph if it exists. If not, it will return some similar entities to search.
- (2) Lookup[keyword], which returns the next sentence containing keyword in the current passage.
- (3) Finish[answer], which returns the answer and finishes the task.

Here are some examples.

33% accuracy with
GPT-3.5 on a multi-
hop QA task

Question: What is the elevation range for the area that the eastern sector of the Colorado orogeny extends
Thought 1: I need to search Colorado orogeny, find the area that the eastern sector of the Colorado orogeny elevation range of the area.

Action 1: Search[Colorado orogeny]

Observation 1: The Colorado orogeny was an episode of mountain building (an orogeny) in Colorado and
Thought 2: It does not mention the eastern sector. So I need to look up eastern sector.

Action 2: Lookup[eastern sector]

Observation 2: (Result 1 / 1) The eastern sector extends into the High Plains and is called the Central Pl

[... truncated ...]

Multi-Hop Retrieval-Augmented Generation (HotPotQA)

Program	Optimized	GPT 3.5	Llama2-13b-Chat
<code>dspy.Predict("question -> answer")</code>	✗	34.3	27.5
<code>dspy.RAG (with CoT)</code>	✗	36.4	34.5
	✓	42.3	38.3
MultiHop	✗	36.9	34.7
	✓	54.7	50.0

Compiling MultiHop into a small LM (T5-770M) with DSPy's BootstrapFinetune, starting from 200 answers, scores 39%

SLMs...



Small Language Models are the Future of Agentic AI

**Peter Belcak¹ Greg Heinrich¹ Shizhe Diao¹ Yonggan Fu¹ Xin Dong¹
Saurav Muralidharan¹ Yingyan Celine Lin^{1,2} Pavlo Molchanov¹**

¹NVIDIA Research ²Georgia Institute of Technology

agents-research@nvidia.com

Optimizers

DSPy Optimizers vary in how they tune the prompts & weights in a program, but at a high level they typically...

- Construct an initial prompt from each module via an Adapter
- Generate examples of every module via rejection sampling
- Use the examples to update the program's modules
 - Automatic few-shot prompting:
`dspy.BootstrapFewShotWithRandomSearch`
 - Induction of instructions: `dspy.MIPROv2`
 - Multi-stage fine-tuning: `dspy.BootstrapFinetune`

Optimizing Instructions and Demonstrations for Multi-Stage Language Model Programs

Krista Opsahl-Ong^{1*}, Michael J Ryan^{1*}, Josh Purtell²,
David Broman³, Christopher Potts¹, Matei Zaharia⁴, Omar Khattab¹

¹Stanford University, ²Basis, ³KTH Royal Institute of Technology ⁴UC Berkeley

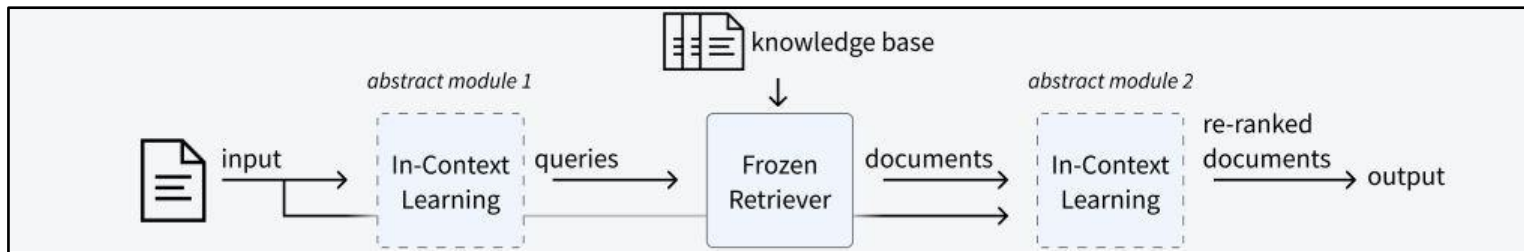
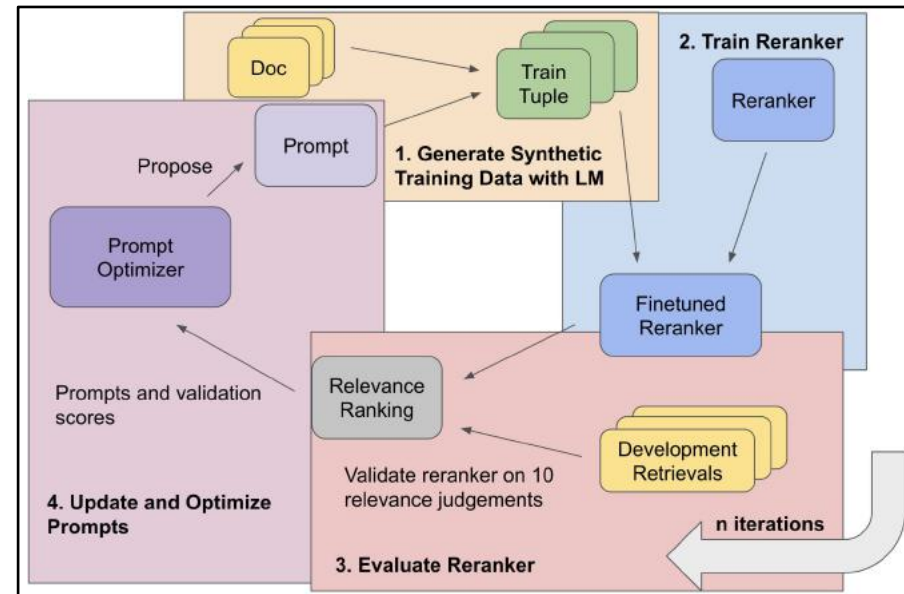
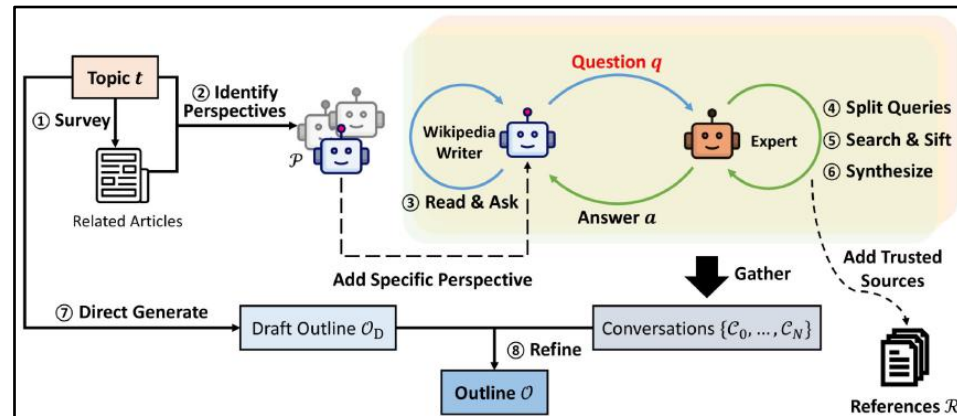
Fine-Tuning and Prompt Optimization: Two Great Steps that Work Better Together

Dilara Soylu Christopher Potts Omar Khattab

Stanford University

Enabled many SoTA systems

like PATH (Jasper Xia, UWaterloo); IReRa (Karel D'Oosterlink, UGhent), STORM (Yijia Shao, Stanford), EDEN (Siyan Li, Columbia), Efficient Agents (Sayash Kapoor, Princeton), ECG-Chat (Yubao Zhao, Beijing Normal U), ...

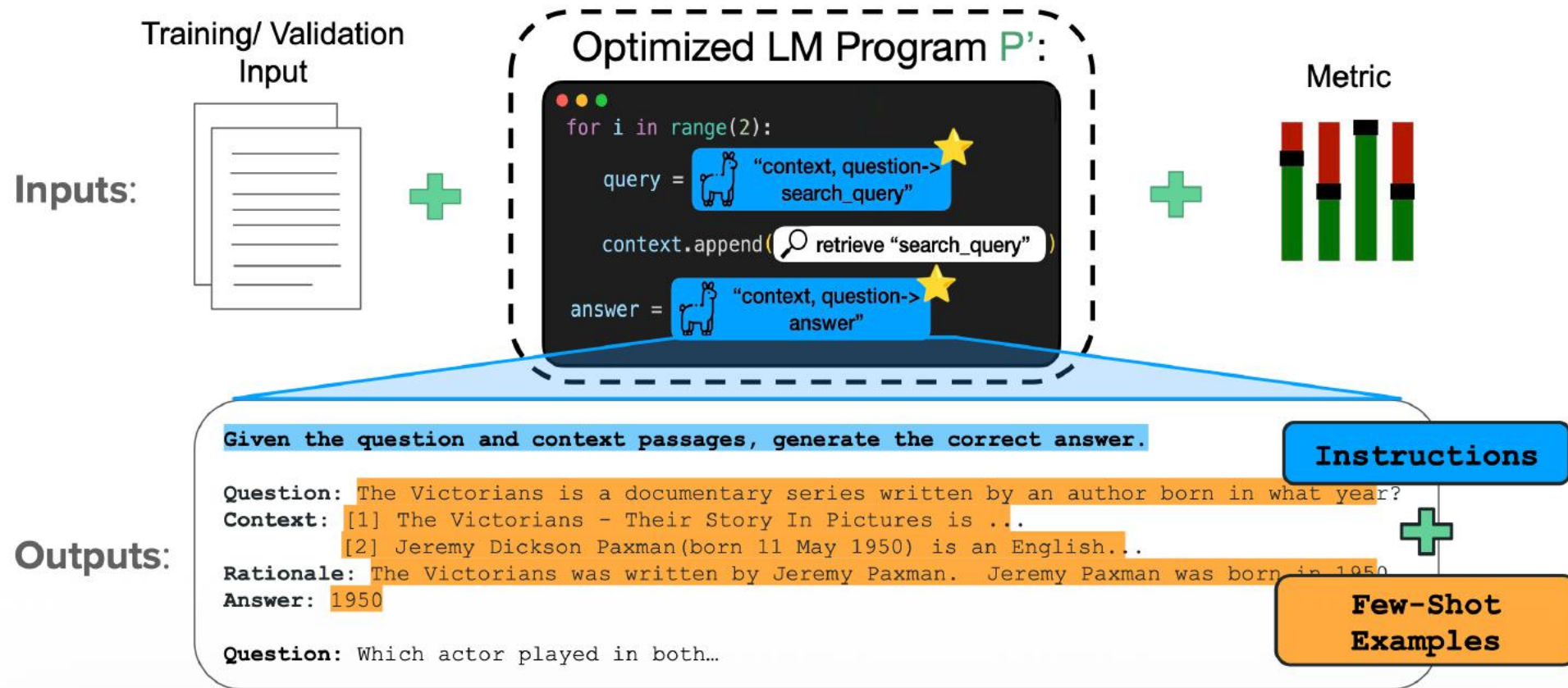


Optimizing Instructions and Demonstrations for Multi-Stage Language Model Programs

**Krista Opsahl-Ong^{1*}, Michael J Ryan^{1*}, Josh Purtell²,
David Broman³, Christopher Potts¹, Matei Zaharia⁴, Omar Khattab¹**

¹Stanford University, ²Basis, ³KTH Royal Institute of Technology ⁴UC Berkeley

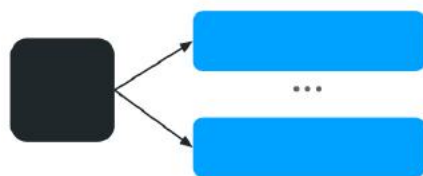
Problem Setting



Constraints / Assumptions

1. **No access to log-probs or model weights:**
Developers may want to optimize LM programs for use on API only models.
2. **No intermediate metrics / labels:** We assume no access to manual ground-truth labels for intermediate stages.
3. **Budget-Conscious:** We want to limit the number of input examples we require and the number of program calls we make.

Key challenges



Prompt Proposal:

Searching over all possible strings is intractable, especially as we add in multiple modules we need to optimize. Instead, we need to propose a small set of high quality options.



Credit Assignment:

We need efficient ways of inferring how each prompt variable contributes to performance, so that we can find the best set for our program.

Bootstrap Few-Shot Examples

LM Program:

```
for i in range(2):  
    query = llama("context, question->  
                  search_query")  
    context.append((🔍 retrieve "search_query"))  
    answer = llama("context, question->  
                   answer")
```

Bootstrap Few-shot examples with simple rejection sampling

Bootstrap Few-Shot Examples

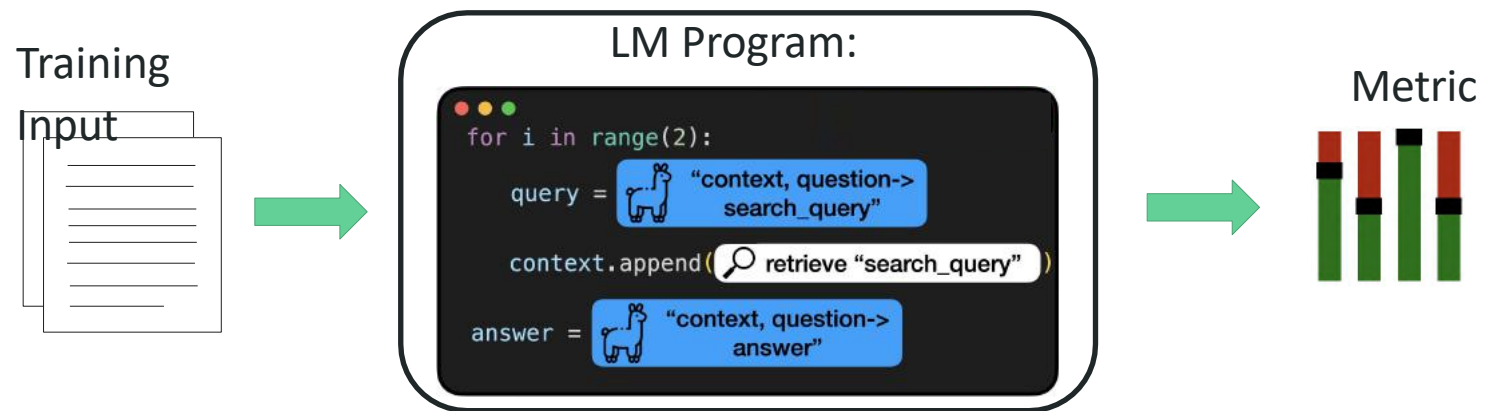
Training
Input



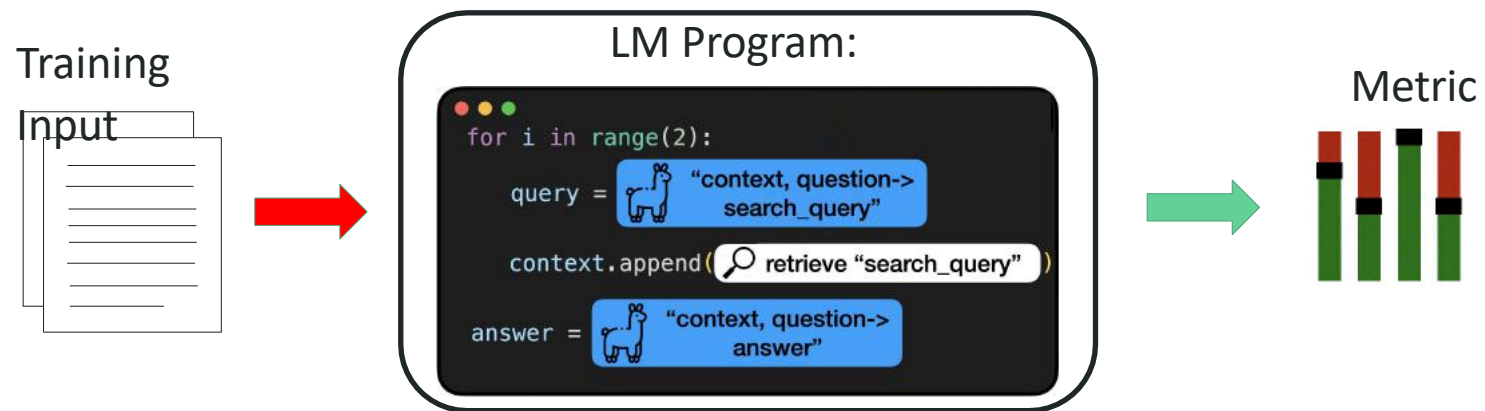
LM Program:

```
for i in range(2):  
    query = llama("context, question->  
                  search_query")  
    context.append((search("retrieve \"search_query\""))  
    answer = llama("context, question->  
                   answer")
```

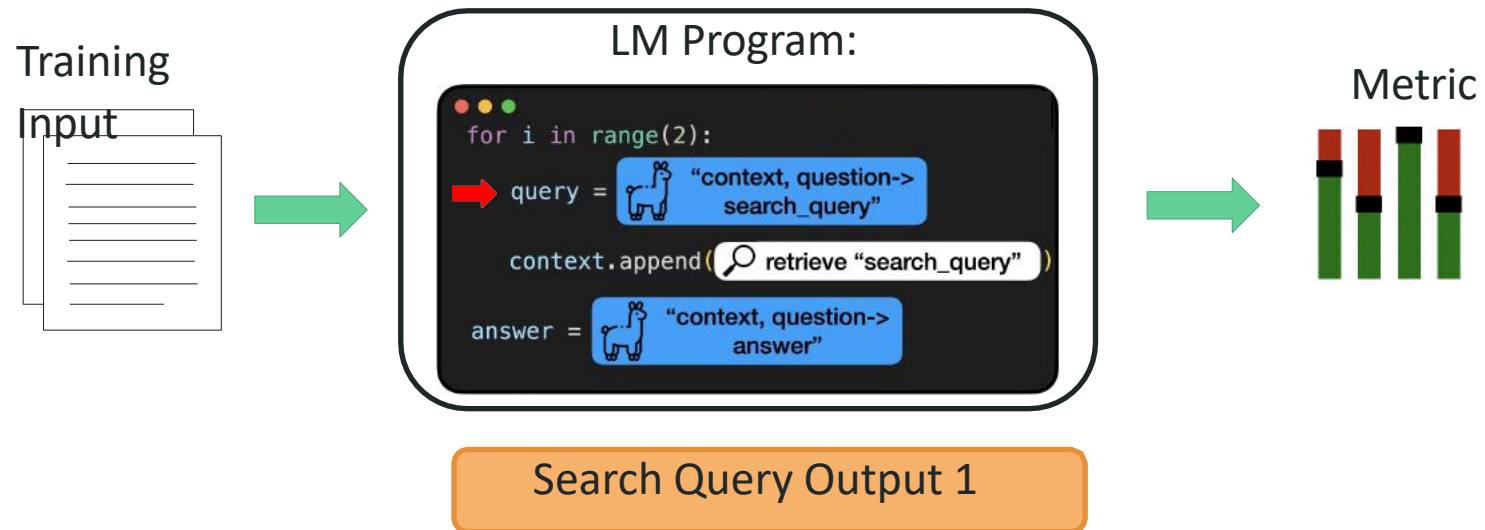
Bootstrap Few-Shot Examples



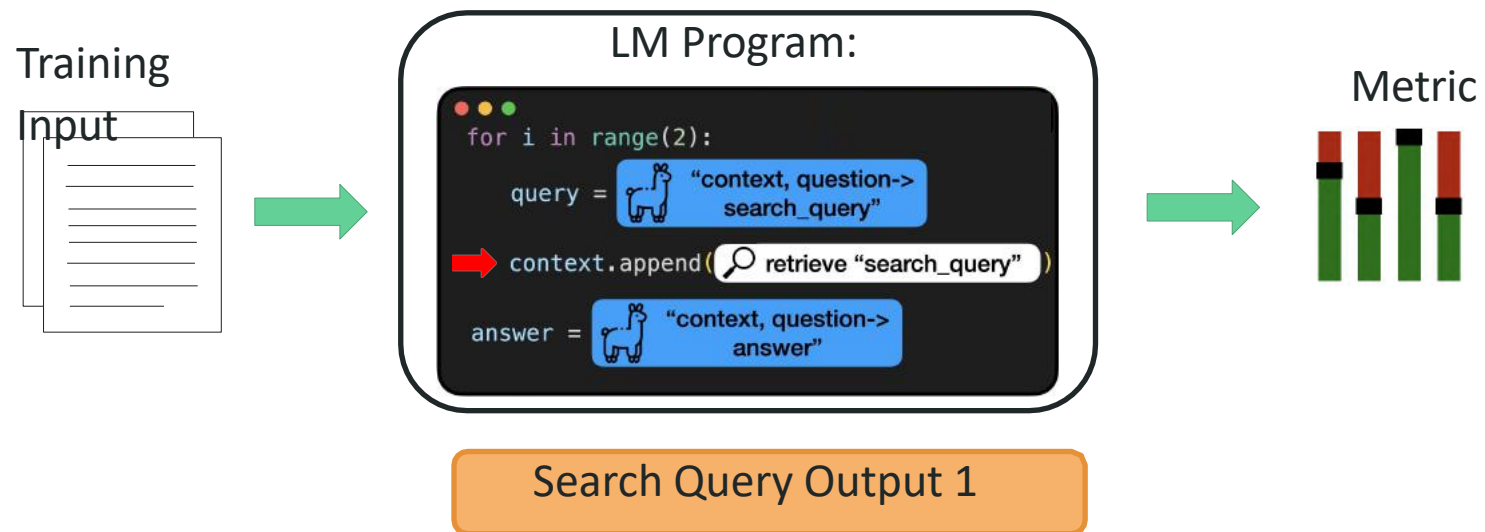
Bootstrap Few-Shot Examples



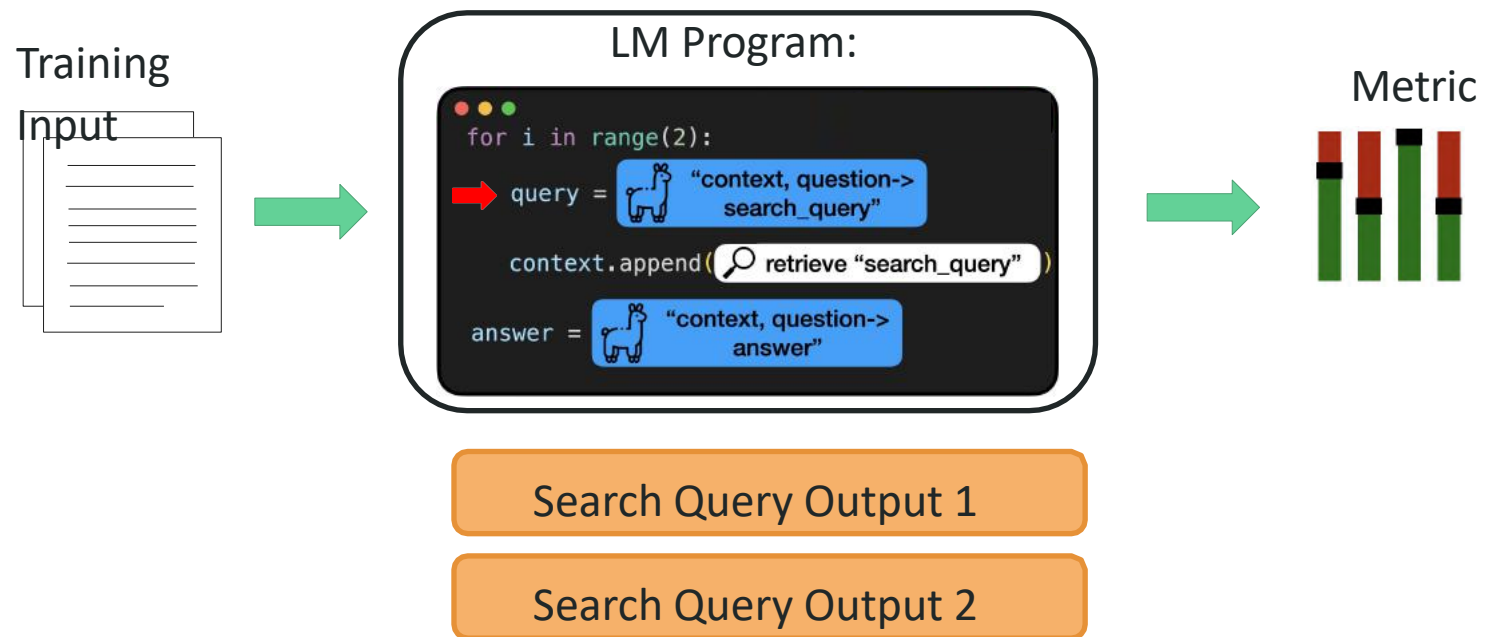
Bootstrap Few-Shot Examples



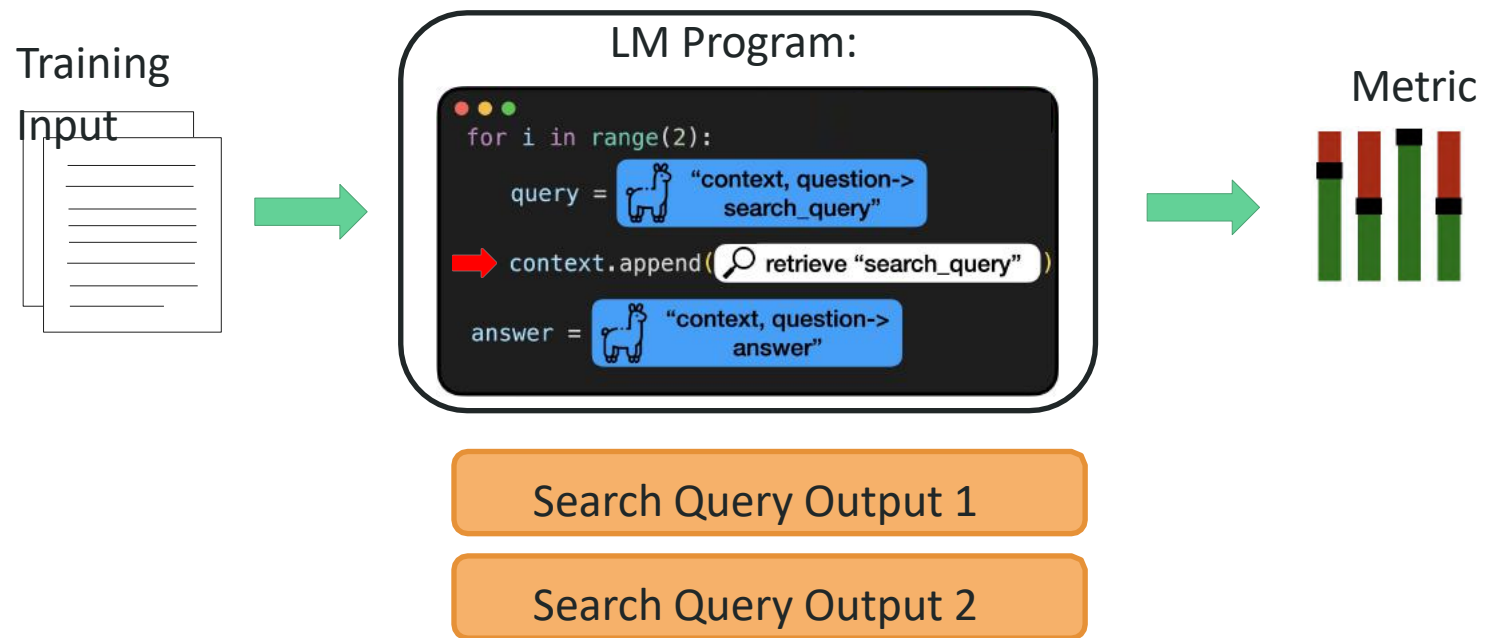
Bootstrap Few-Shot Examples



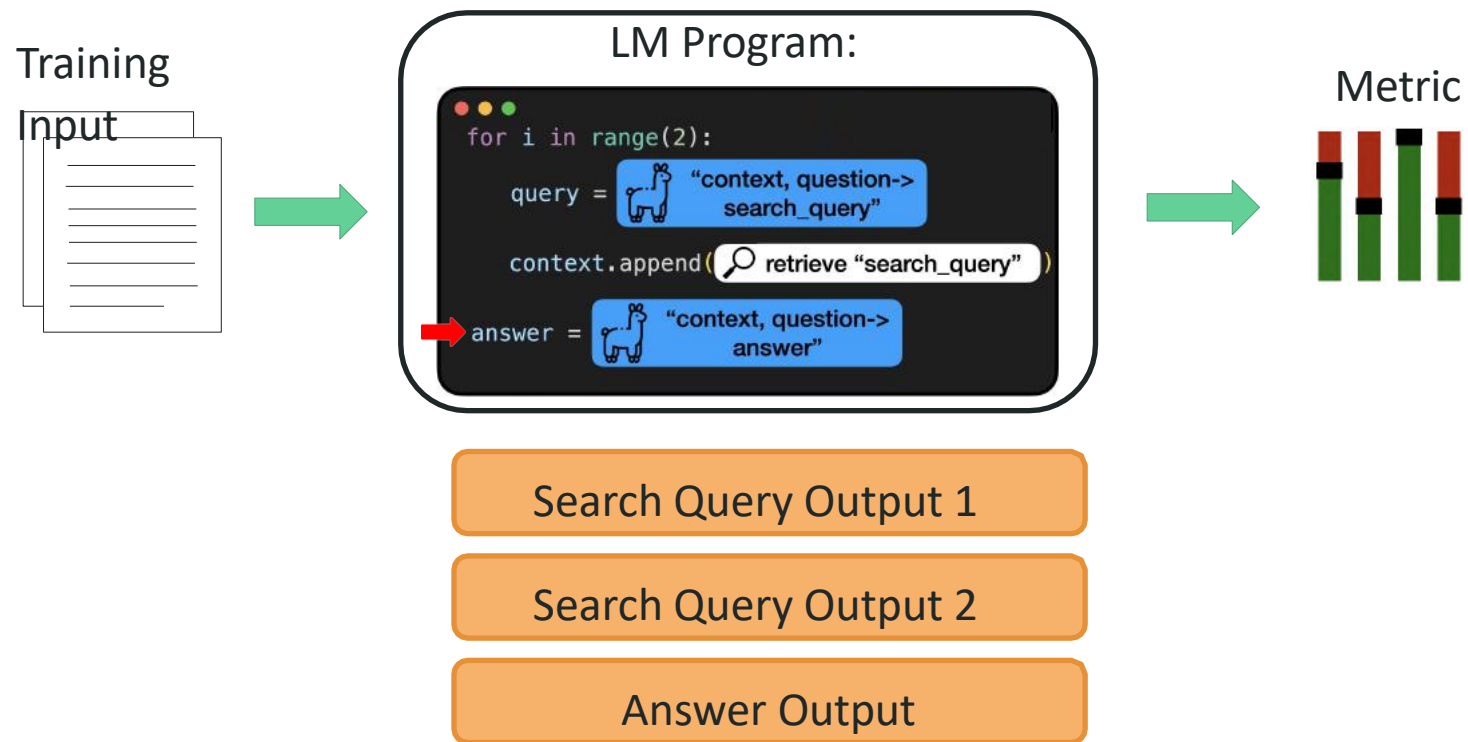
Bootstrap Few-Shot Examples



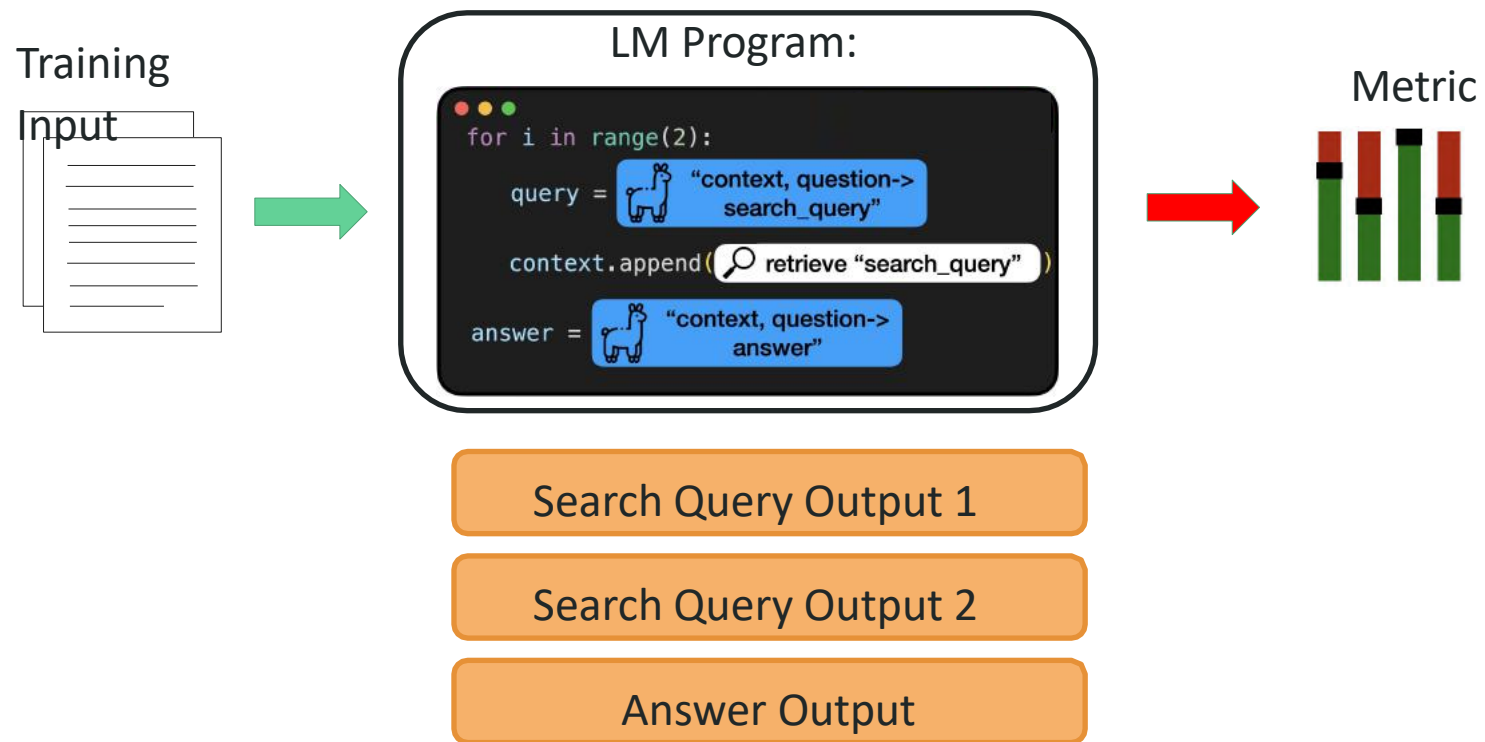
Bootstrap Few-Shot Examples



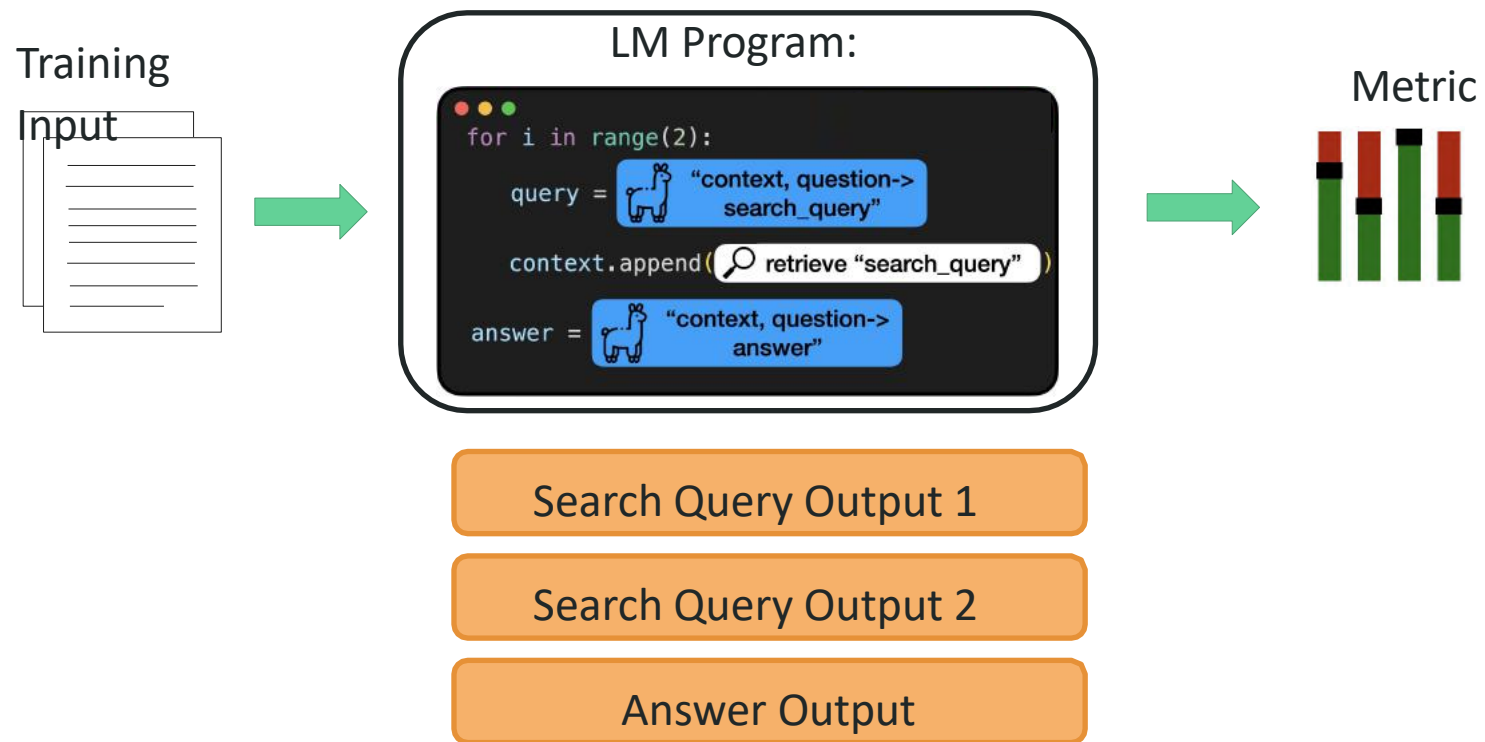
Bootstrap Few-Shot Examples



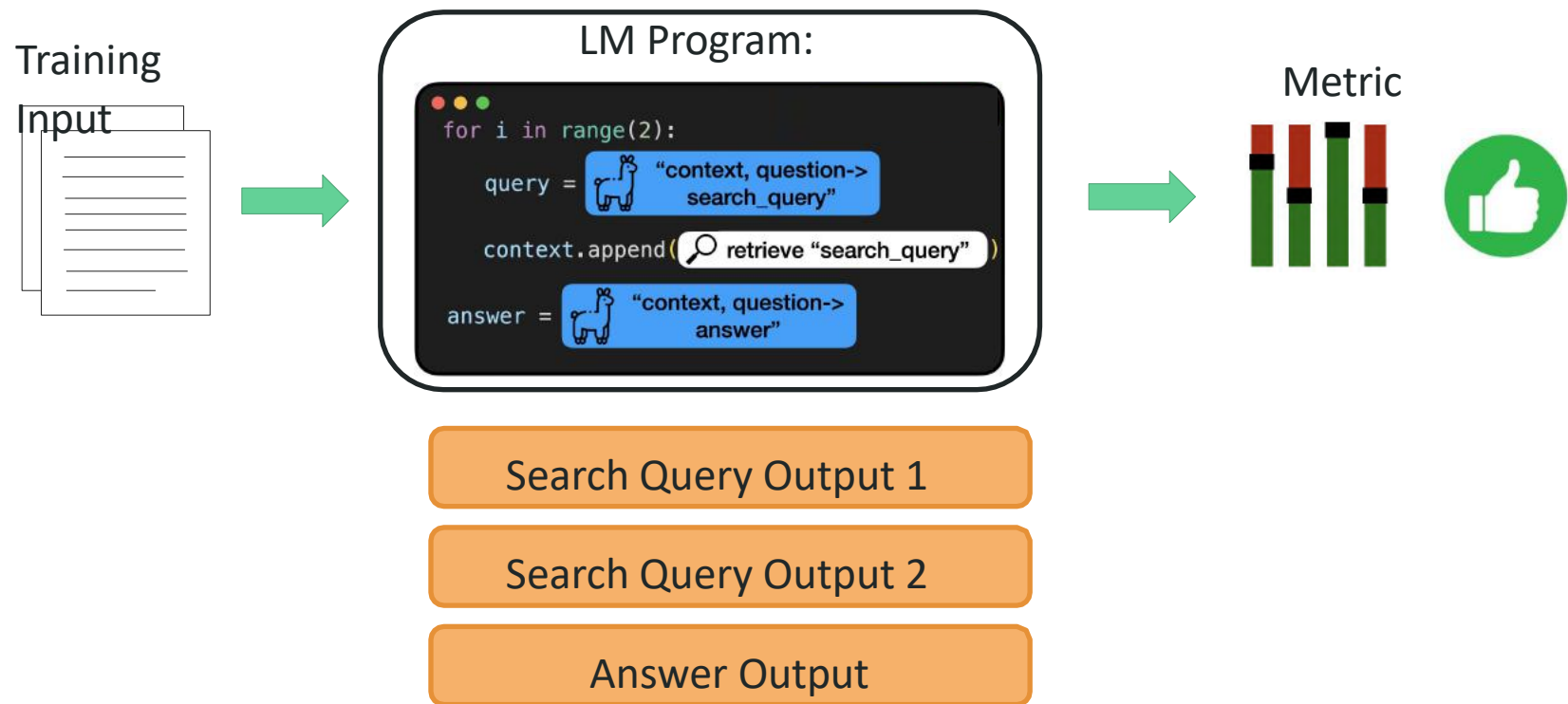
Bootstrap Few-Shot Examples



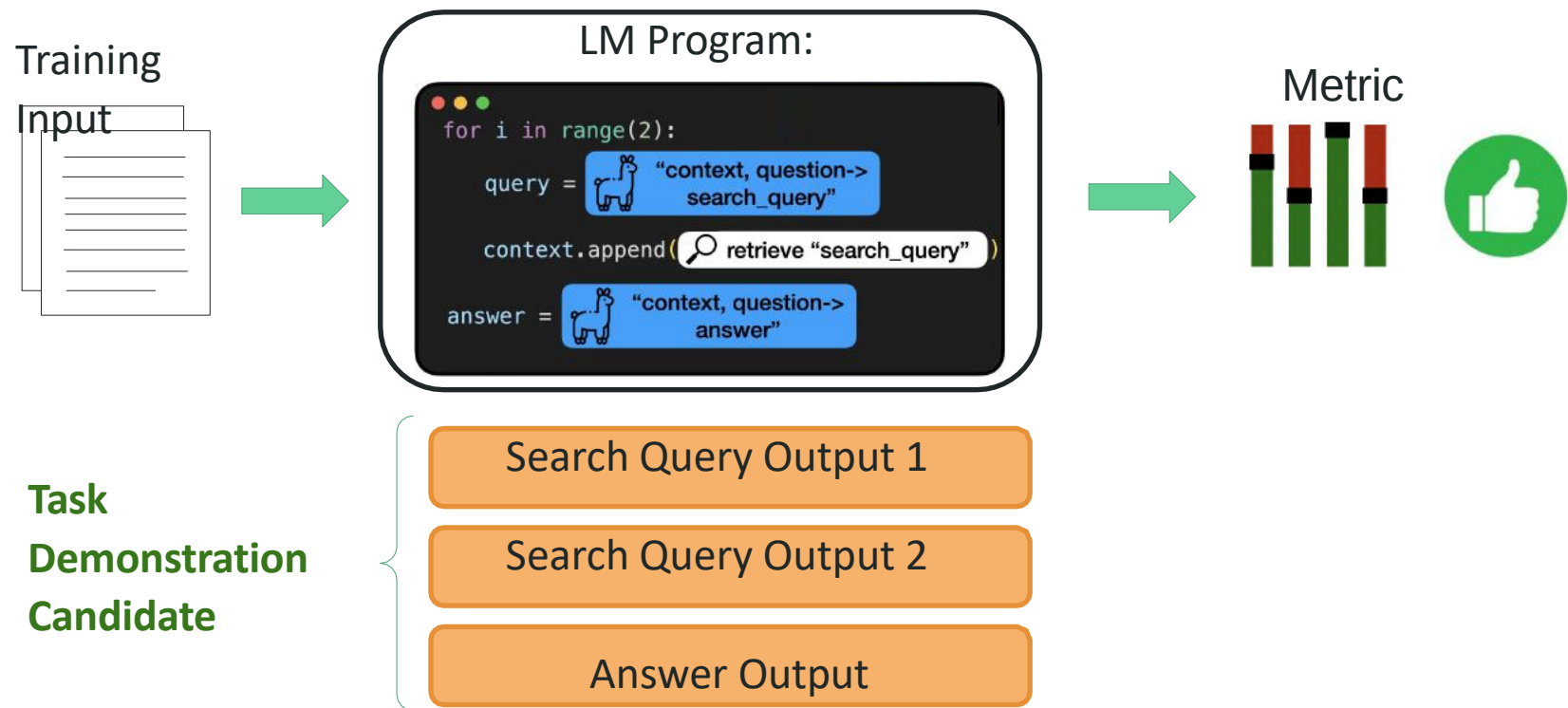
Bootstrap Few-Shot Examples



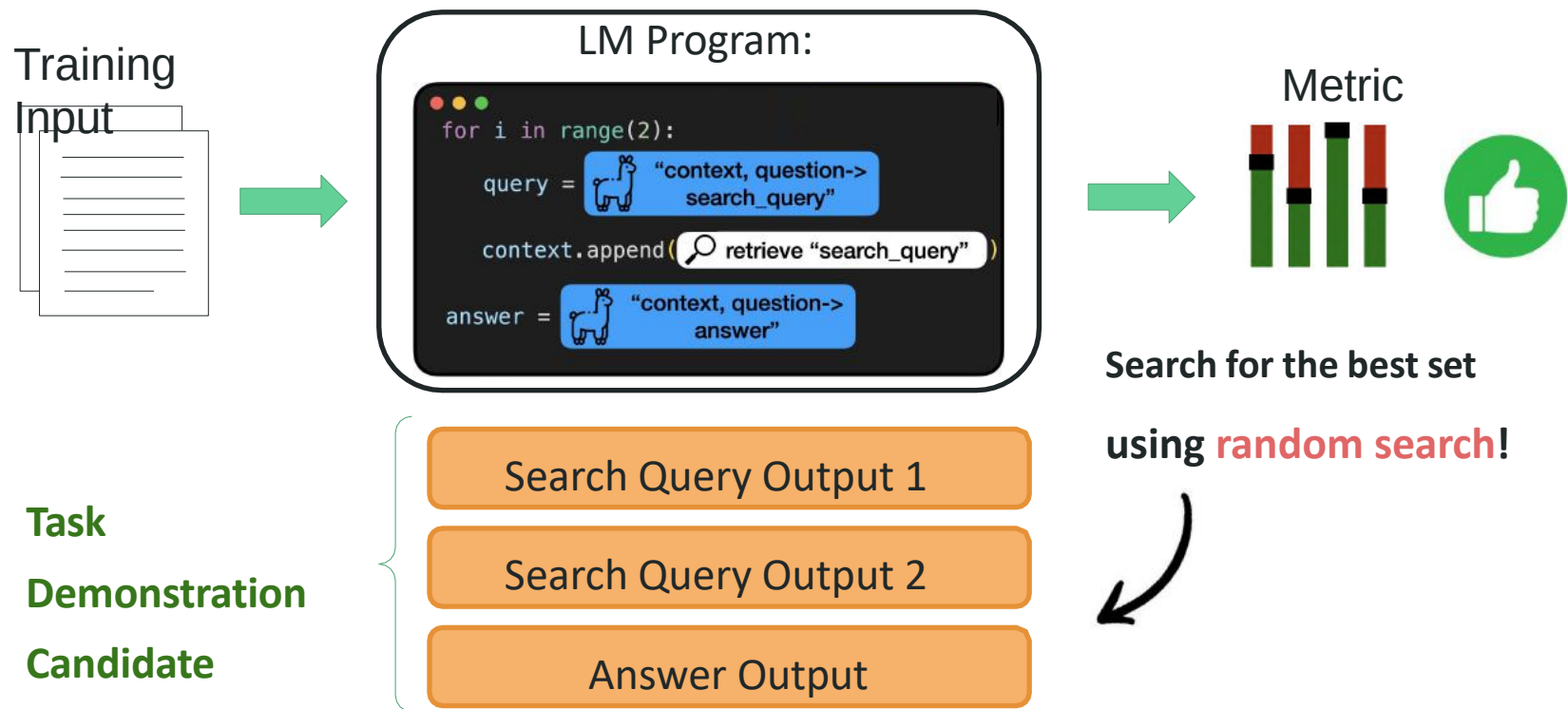
Bootstrap Few-Shot Examples



Bootstrap Few-Shot Examples



Bootstrap Few-Shot (w/ Random Search)



Bootstrap Few-Shot (w/ Random Search)

Given the context passages and a question, generate the correct answer.

Context: [1] The Victorians - Their Story In Pictures is ...

[2] Jeremy Dickson Paxman(born 11 May 1950) is an English...

Question: The Victorians is a documentary series written by an author born in what year?

Rationale: The Victorians was written by Jeremy Paxman. Jeremy Paxman was born in 1950.

Answer: 1950

...

Task

Demonstration

Candidate

Search Query Output 1

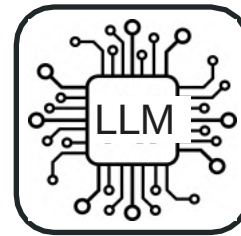
Search Query Output 2

Answer Output

What is OPRO?

Optimization through Prompting

“Proposer LM”



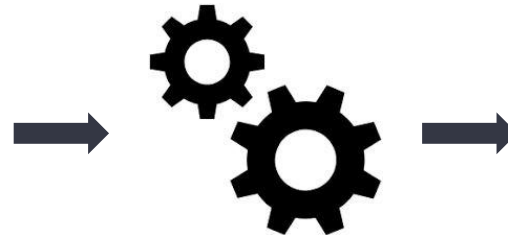
Prompt Proposals

“Think step by step”

“Take a deep breath and
think step by step”

“Carefully solve the
problem”

“Let’s do the math”



Evaluate

Propose More Prompts

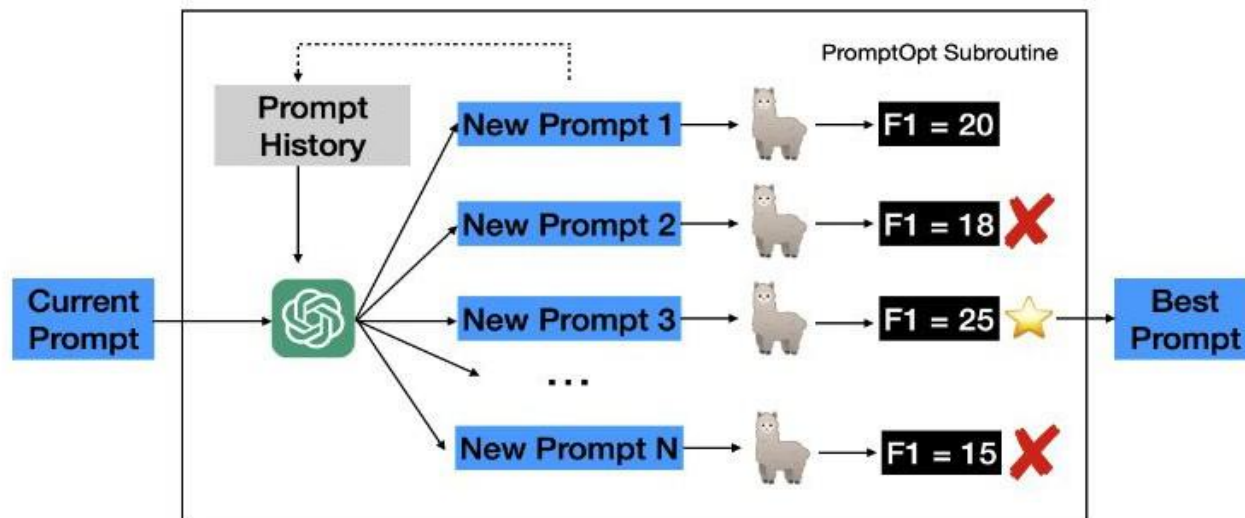
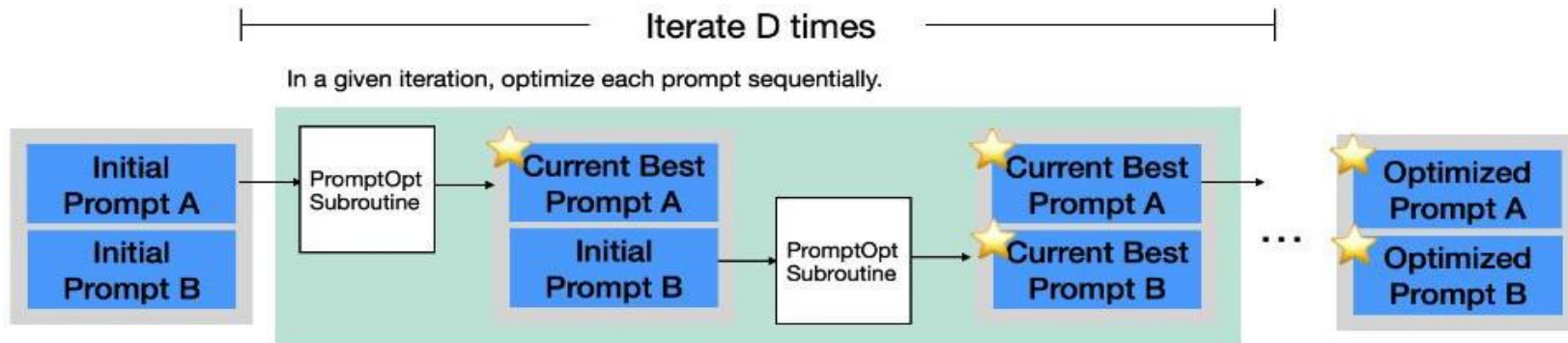
Given prompts/scores
propose more prompts.

“Think step by step”
Score: 31

“Take a deep breath
and think step by step”
Score: 42

Initial extension to multi-stage: CA-OPRO

Coordinate-Ascent OPRO



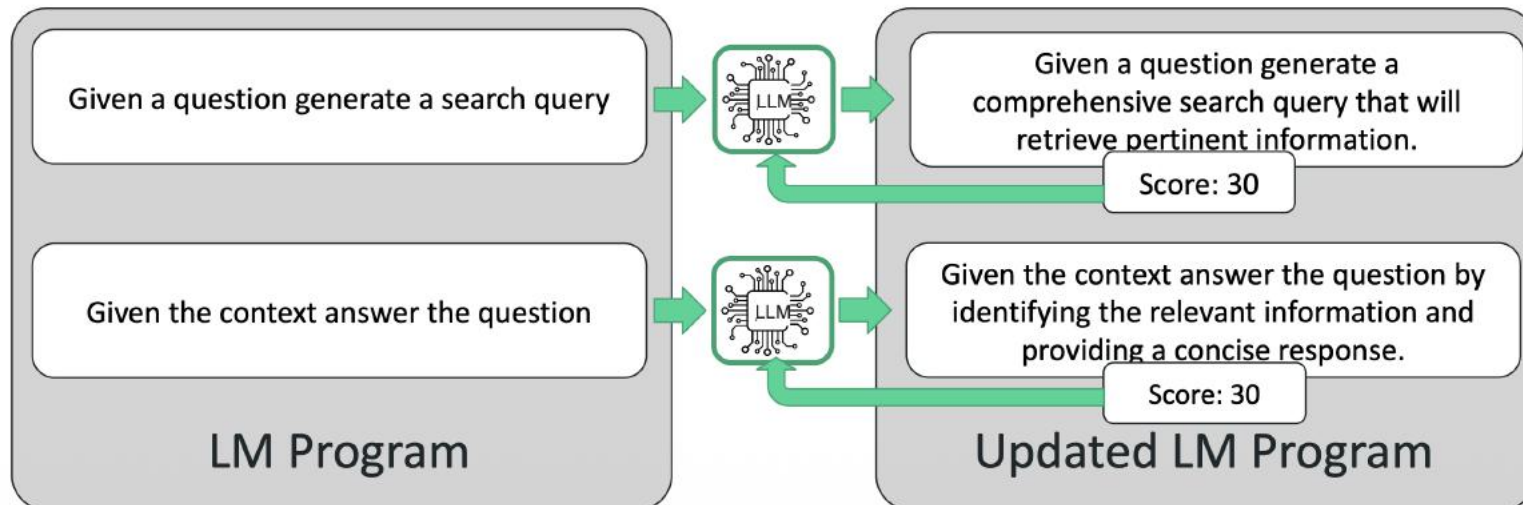
Module-Level OPRO



Key Idea: Coordinate-Ascent was expensive, maybe we don't need explicit credit assignment? Let's just change both prompts at a time in parallel!

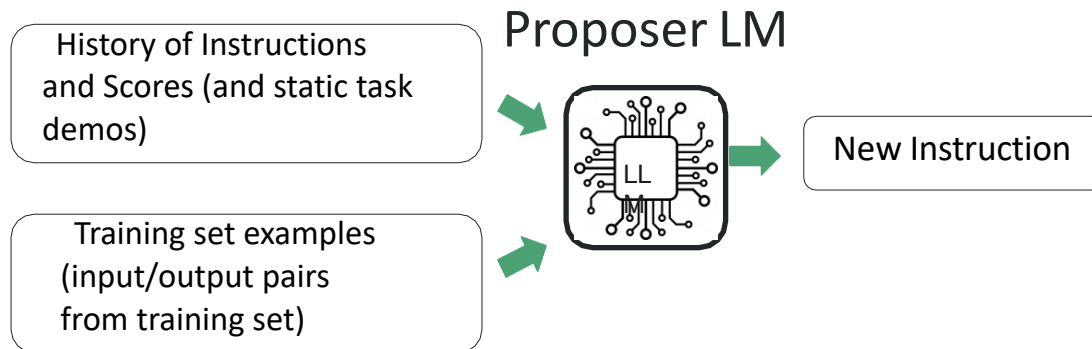
Module-Level OPRO

Key Idea: Coordinate-Ascent was expensive, maybe we don't need explicit credit assignment? Let's just change both prompts at a time in parallel!



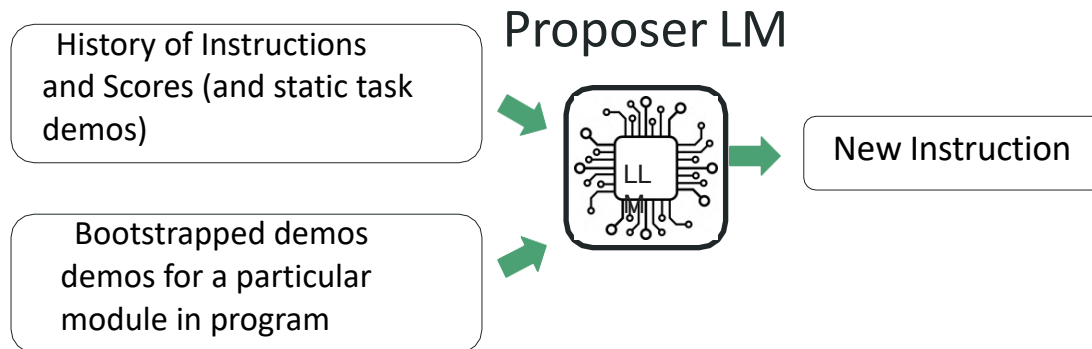
Grounding!

Hypothesis: Providing our proposer LM with more information relevant to the task can help us propose better instructions.



Finally, Grounding!

Hypothesis: Providing our proposer LM with more information relevant to the task can help us propose better instructions.



Bootstrapped demo example:

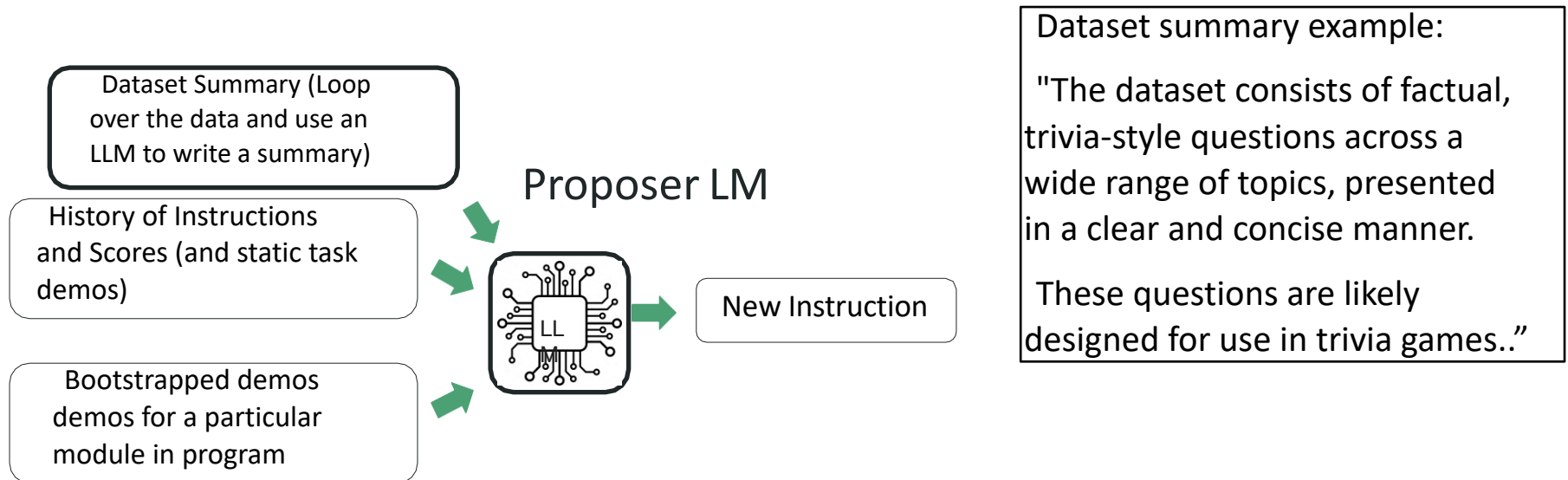
Question: The Victorians - Their Story In Pictures is a documentary series written by an author born in what year?

Reasoning: Let's think step by step in order to find the search query. We need to find the author's birth year. We can search for the author's name along with the phrase "birth year" or "birthday" to get the desired information.

- Their Story In Pictures birth year"
- Their Story In Pictures birthday"

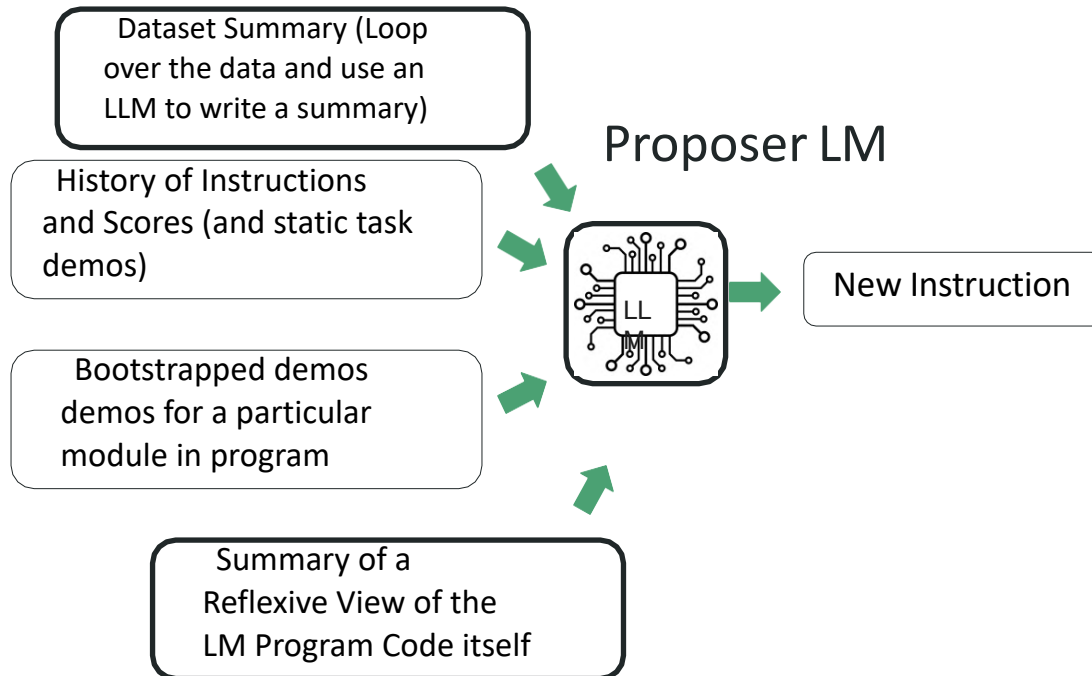
Finally, Grounding!

Hypothesis: Providing our proposer LM with more information relevant to the task can help us propose better instructions.



Finally, Grounding!

Hypothesis: Providing our proposer LM with more information relevant to the task can help us propose better instructions.



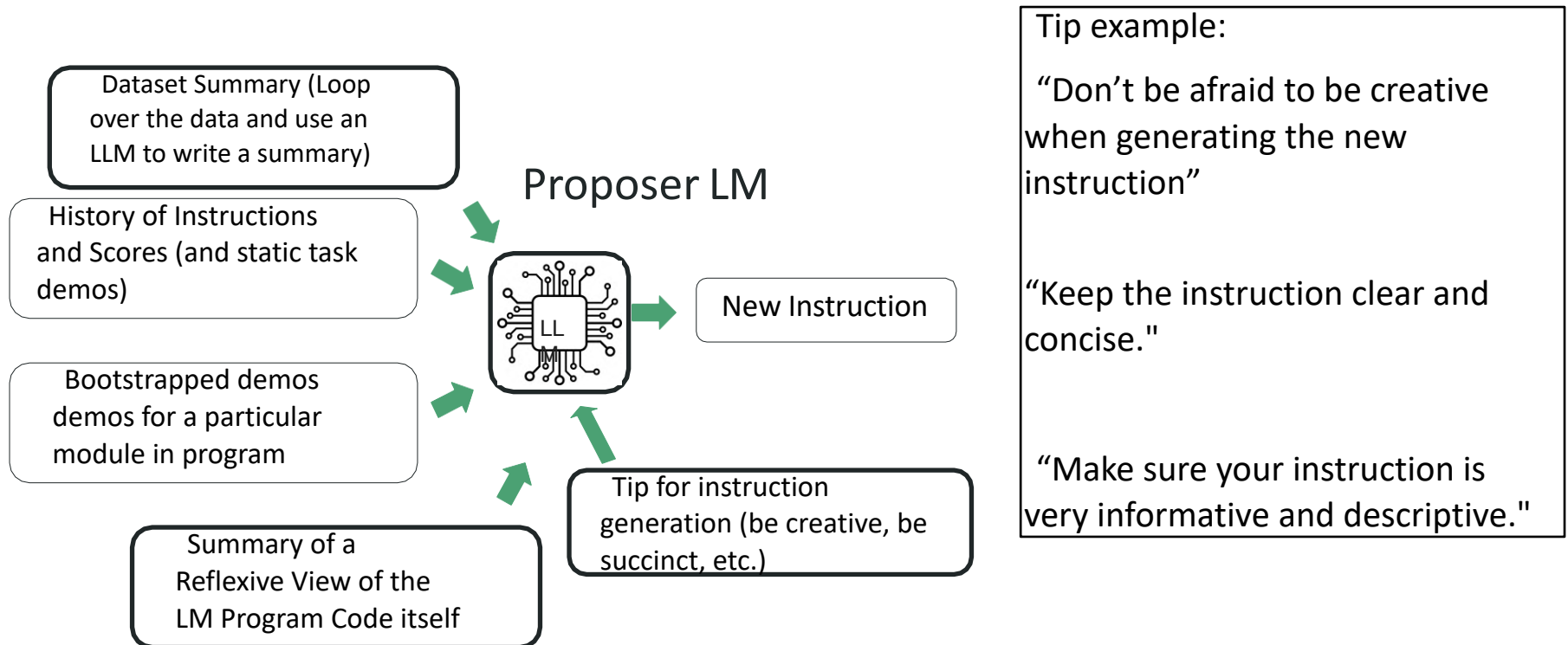
Program Summary example:

“The program code appears to be designed to answer complex questions by retrieving and processing information from multiple sources or passages. In this case, the program is set up for two hops, ... The module

``self.generate_query`` in this program is responsible for generating a search query based on the context and question provided.”

Finally, Grounding!

Hypothesis: Providing our proposer LM with more information relevant to the task can help us propose better instructions.



Multi-prompt Instruction PProposal Optimizer

Prompt Proposal

1. Bootstrap Task Demonstrations
2. Propose Instruction Candidates using an LM Program

Credit Assignment

3. Jointly tune with a Bayesian hyperparameter optimizer

Summary

- Monolithic LMs are powerful but unreliable
 - → hard to control, debug, and improve
- Compound AI Systems
 - = modular programs using LMs as components
 - Key benefits: quality, control, transparency, efficiency, inference-time scaling
 - Shift from prompt engineering → programming LM systems
- DSPy framework abstracts LM pipelines into structured modules (signatures, modules, optimizers)
 - Automatically generates & optimizes prompts instead of manual trial-and-error
 - Uses techniques like bootstrapping, few-shot learning, and optimizers (e.g., MIPRO)
 - Enables systematic, scalable, and reliable AI development